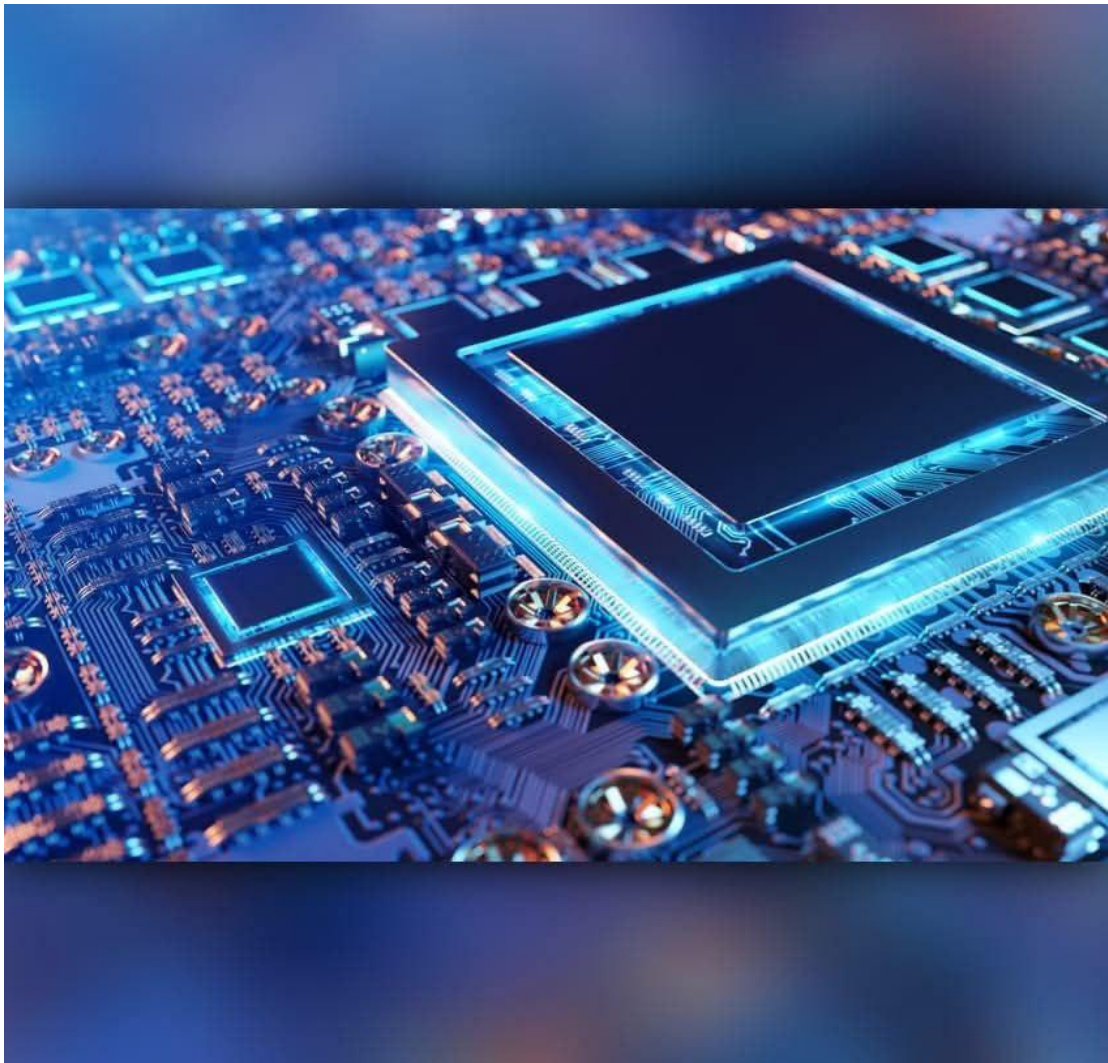


Assignment_One

Prepared by :
Omar Ahmed Abdelaty



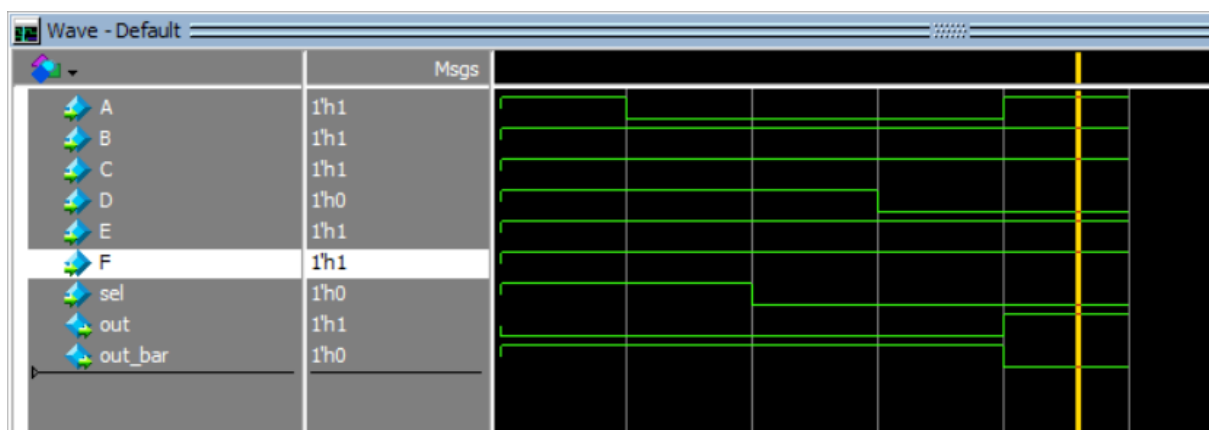
Submitted to: Eng. Kareem Waseem

Question_1

Code

```
1  module q_one (  
2      input wire A, B, C, D, E, F, sel,  
3      output wire out, out_bar  
4  );  
5  
6      wire and1_out, xnor1_out;  
7  
8      assign and1_out = A & B & C;  
9      assign xnor1_out = ~(D ^ E ^ F);  
10     assign out = sel ? xnor1_out : and1_out;  
11     assign out_bar = ~out;  
12  
13 endmodule
```

Simulation

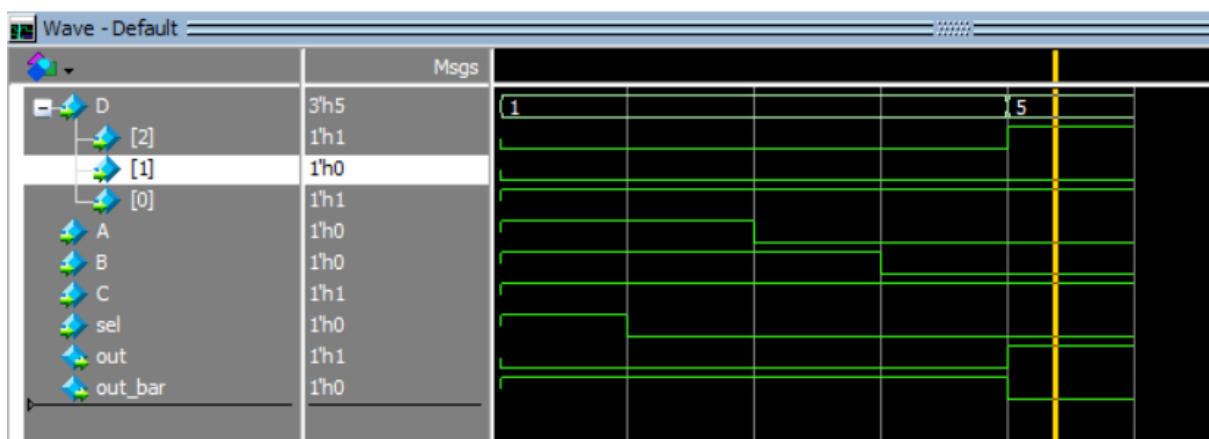


Question_2

Code

```
1  module q_two (  
2      input wire [2:0] D,  
3      input wire A, B, C, sel,  
4      output reg out, out_bar  
5  );  
6  
7      always @(*) begin  
8          case (sel)  
9              1'b0: out = (D[0] & D[1]) | D[2];  
10             1'b1: out = ~(A ^ B ^ C);  
11          endcase  
12          out_bar = ~out;  
13      end  
14  
15  endmodule
```

Simulation

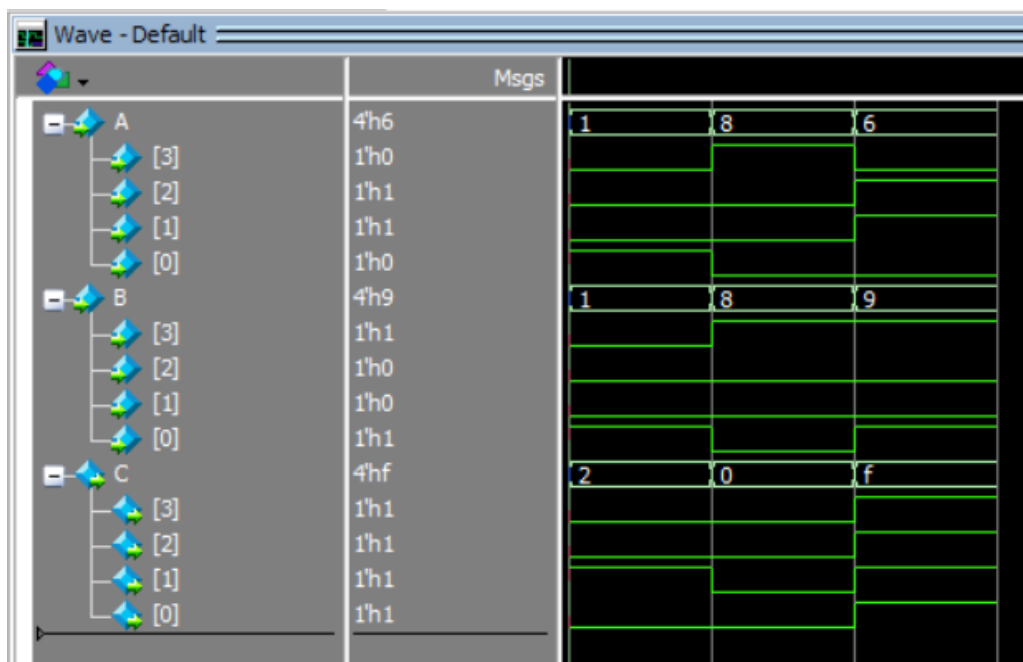


Question_3

Code

```
1  module four_bit_adder (  
2      input wire [3:0] A,  
3      input wire [3:0] B,  
4      output wire [3:0] C  
5  );  
6  
7      assign C = A + B;  
8  
9  endmodule
```

Simulation

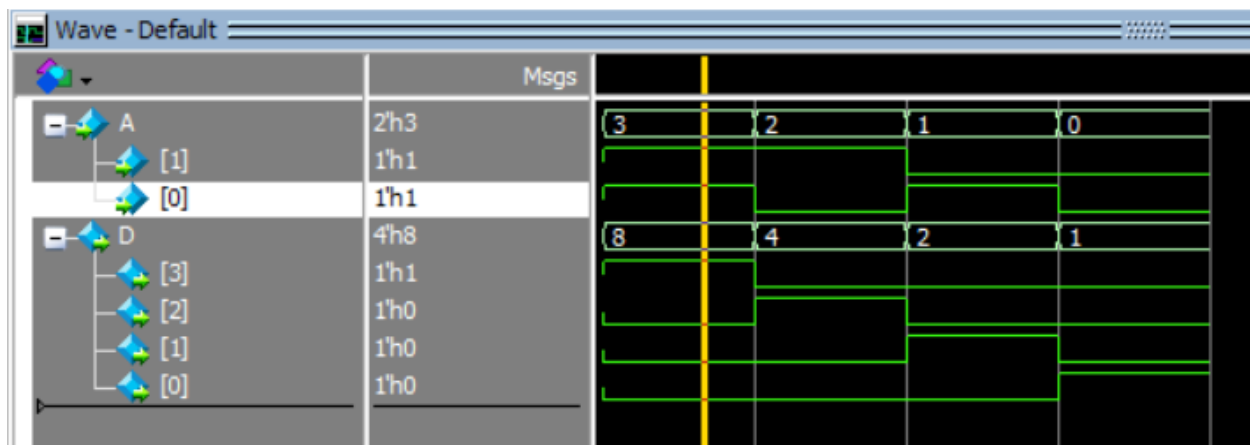


Question_4

Code

```
1  module two_to_four_decoder (  
2      input wire [1:0] A,  
3      output wire [3:0] D  
4  );  
5  
6      assign D = (A == 2'b00) ? 4'b0001 :  
7                  (A == 2'b01) ? 4'b0010 :  
8                  (A == 2'b10) ? 4'b0100 :  
9                  (A == 2'b11) ? 4'b1000 : 4'b0000;  
10  
11  endmodule
```

Simulation



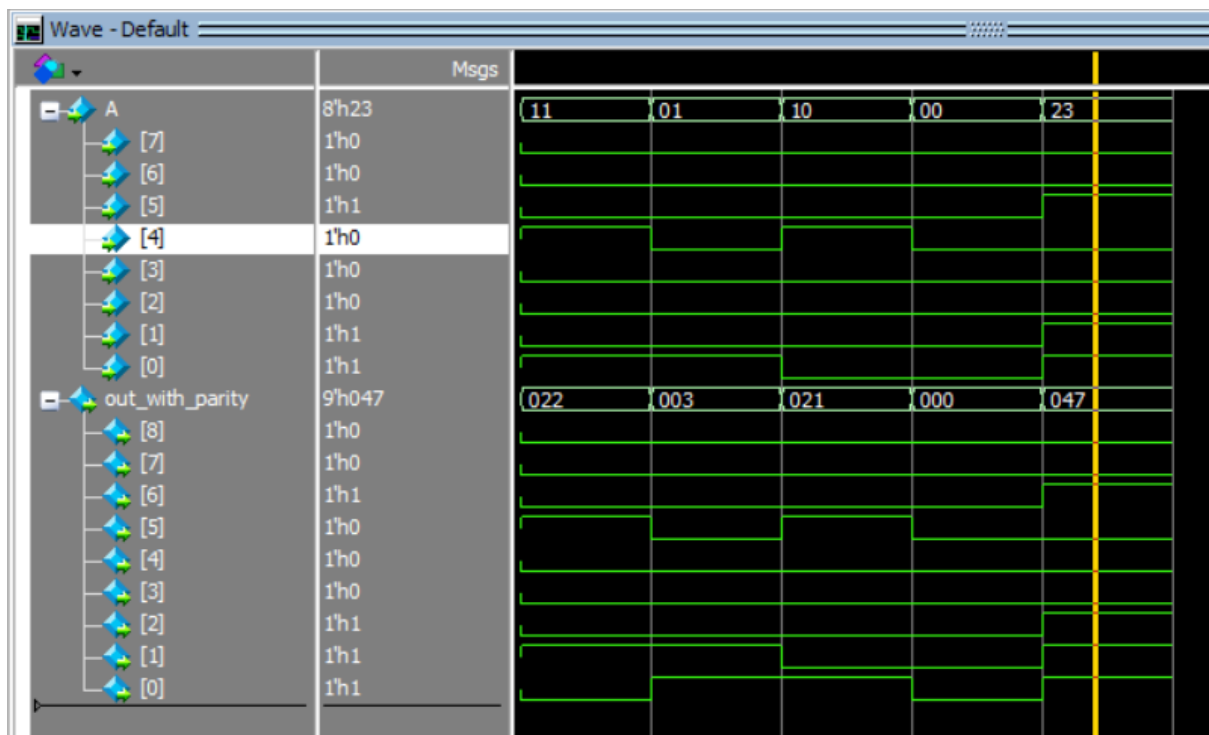
Question_5

Code

```
1 module even_parity_generator (  
2     input wire [7:0] A,  
3     output wire [8:0] out_with_parity  
4 );  
5  
6     assign out_with_parity = {A, ^A};  
7  
8 endmodule
```

Xor as when number of 1's in A is even so parity bit=0 and if odd parity bit =1

Simulation



Question_6

Code of ALU

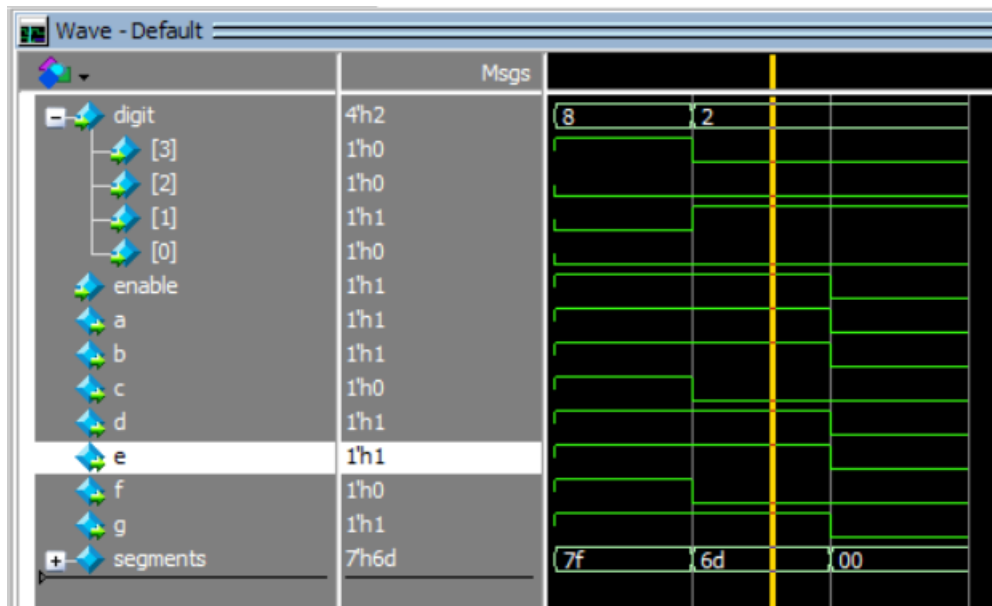


```
1  module ALU(in0,in1,opcode,out);
2  parameter N=4;
3  input [N-1:0]in0,in1;
4  input [1:0] opcode;
5  output reg [N-1:0]out ;
6
7  always @(*) begin
8      case (opcode)
9          2'b00: out=in0 + in1;
10         2'b01: out=in0 | in1;
11         2'b10: out=in0 - in1;
12         2'b11: out=in0 ^ in1 ;
13
14         default: out =2'b00;
15     endcase
16 end
17 endmodule
```

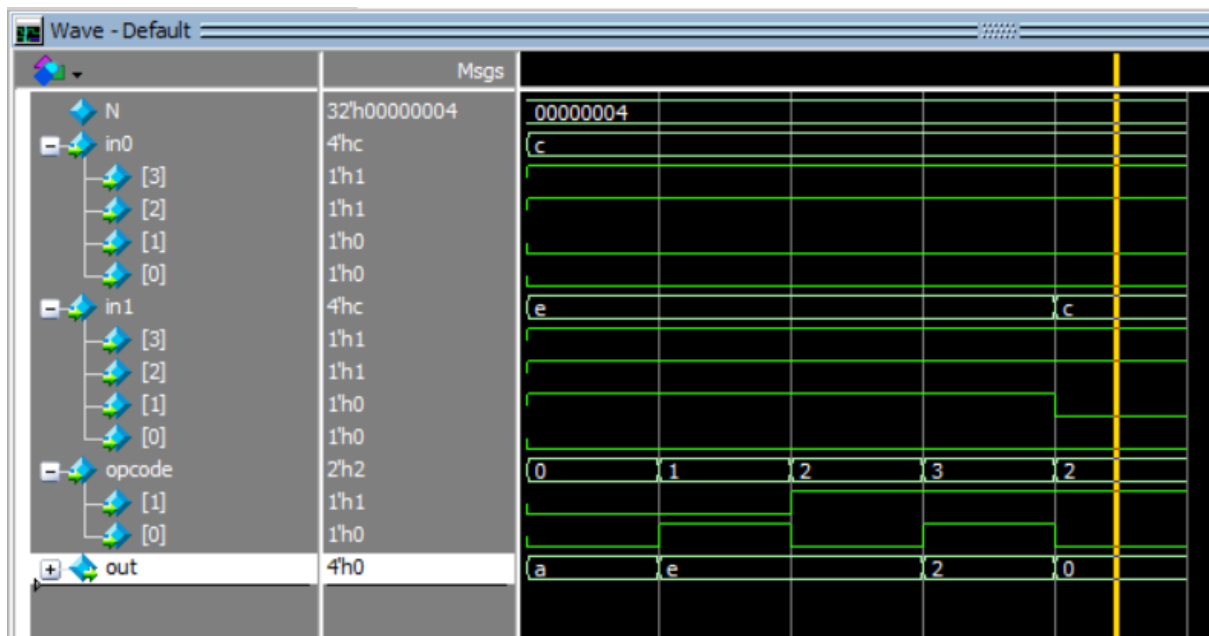
Code of 7_segment

```
1  module seven_segment_decoder (  
2      input wire [3:0] digit,  
3      input wire enable,  
4      output wire a, b, c, d, e, f, g  
5  );  
6  
7      reg [6:0] segments;  
8  
9      always @(*) begin  
10         if (enable) begin  
11             case (digit)  
12                 4'h0: segments = 7'b1111110;  
13                 4'h1: segments = 7'b0110000;  
14                 4'h2: segments = 7'b1101101;  
15                 4'h3: segments = 7'b1111001;  
16                 4'h4: segments = 7'b0110011;  
17                 4'h5: segments = 7'b1011011;  
18                 4'h6: segments = 7'b1011111;  
19                 4'h7: segments = 7'b1110000;  
20                 4'h8: segments = 7'b1111111;  
21                 4'h9: segments = 7'b1111011;  
22                 4'hA: segments = 7'b1110111;  
23                 4'hB: segments = 7'b0011111;  
24                 4'hC: segments = 7'b1001110;  
25                 4'hD: segments = 7'b0111101;  
26                 4'hE: segments = 7'b1001111;  
27                 4'hF: segments = 7'b1000111;  
28                 default: segments = 7'b0000000;  
29             endcase  
30         end else begin  
31             segments = 7'b0000000;  
32         end  
33     end  
34  
35     assign {a, b, c, d, e, f, g} = segments;  
36  
37 endmodule
```


Simulation 7 segment only



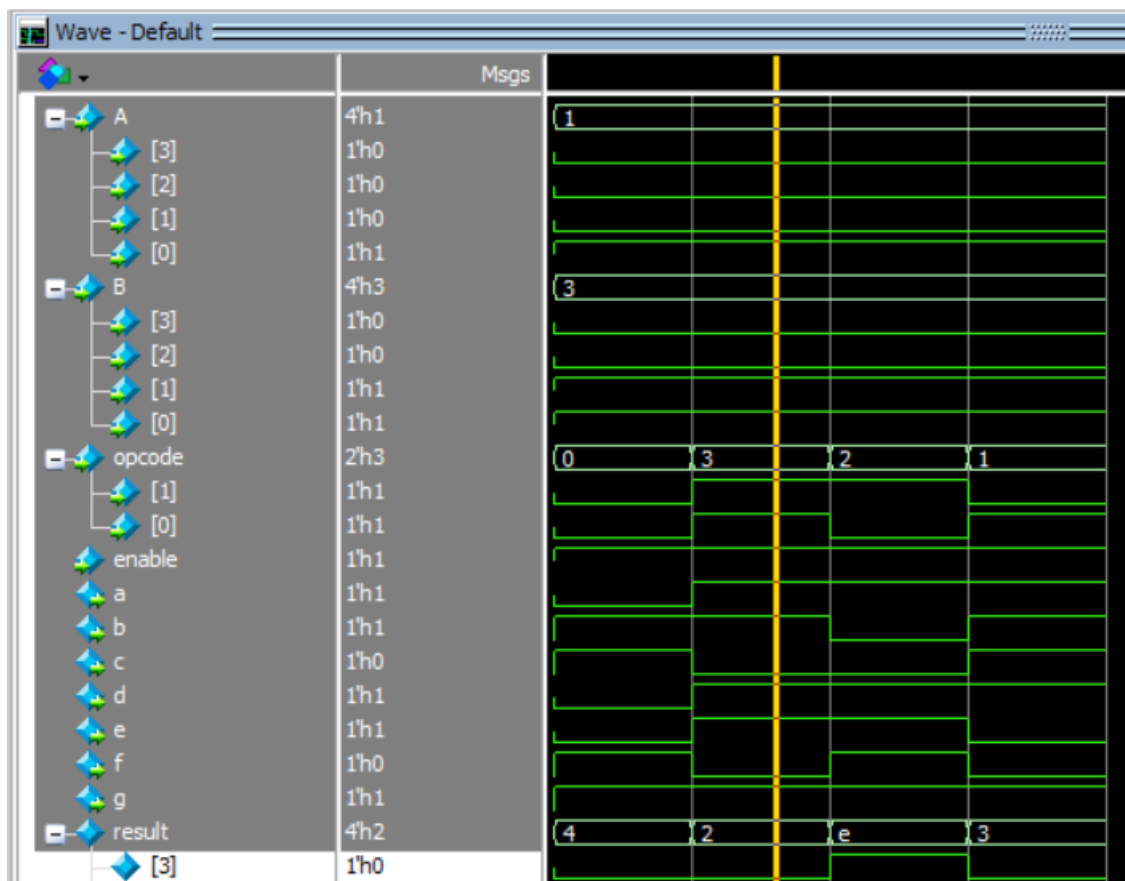
Simulation of ALU only



Code of ALU with 7_Segment

```
1  module alu_7segment_system (  
2      input wire [3:0] A,  
3      input wire [3:0] B,  
4      input wire [1:0] opcode,  
5      input wire enable,  
6      output wire a, b, c, d, e, f, g  
7  );  
8  
9      wire [3:0] result;  
10  
11  
12      ALU #(.N(4)) alu_instance (  
13          .in0(A),  
14          .in1(B),  
15          .opcode(opcode),  
16          .out(result)  
17      );  
18  
19  
20      seven_segment_decoder decoder_instance (  
21          .digit(result),  
22          .enable(enable),  
23          .a(a), .b(b), .c(c), .d(d), .e(e), .f(f), .g(g)  
24      );  
25  
26  endmodule
```

Simulation of ALU with 7_Segment



Proof

If A is 0001 and B is 0011 and opcode is 3 xor then result 0010 which 2.

If subtraction $0001 - 0011 = 0001 + 1101 = 1110$ which e and -2 in 2's complement.